BANKING MANAGEMENT SYSTEM

Technical Report

Visvesvaraya Technological University

Jnana Sangama, Belagavi, Karnataka

AMC Engineering College

AMC Campus, Bannerghatta Main Road, Bengaluru, Karnataka 560083

Developed By	Akshith Peri
USN	1AM24CS013
Guided By	Prof. T K Pradeep Kumar

1. Abstract

The Banking Management System (BMS) is a full-stack web application designed to digitize and streamline the management of banking operations. The system enables users to manage records of employees, customers, bank accounts, transactions, payments, loans, fixed deposits, recurring deposits, beneficiaries, and audit logs through a centralized platform. It follows a three-tier architecture consisting of a frontend web interface, an Express.js backend REST API, and a MySQL relational database. The project demonstrates practical implementation of REST API design, relational database modeling, JWT-based authentication, role-based access control, and dynamic frontend rendering.

2. Introduction

Banking Management Systems are software solutions that automate administrative and transactional tasks in financial institutions. Traditional manual methods of maintaining banking records using paper registers and spreadsheets are prone to errors, data redundancy, and retrieval delays. This project addresses the need for a simple, secure, web-based solution to manage banking data efficiently.

The objectives of this project are:

- To provide a centralized interface for managing employees, customers, accounts, transactions, and loans
- To implement role-based access control for Admin/Employee and Customer portals
- To support automated EMI schedule generation and loan approval workflows
- To offer real-time account balance tracking, transaction history, and audit logging
- To support Fixed Deposits, Recurring Deposits, Beneficiary management, and KYC verification

3. Technologies Used

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (ES6)	User interface and interactivity
Frontend Libs	Font Awesome 6.4, Chart.js, Google Fonts	Icons, charting, typography
Backend	Node.js, Express.js 4.18	REST API server
Auth	bcryptjs, jsonwebtoken (JWT)	Password hashing and token-based auth
Database	MySQL (via mysql2 driver)	Data storage and relational modeling
Middleware	cors, dotenv	Cross-origin access, environment config

Layer	Technology	Purpose
Tools	VS Code, npm, Thunder Client	Development and API testing

4. System Architecture

The system follows a three-tier architecture. The Frontend (Browser) communicates with the Backend (Express.js on port 5000) via HTTP requests carrying JSON payloads and JWT tokens. The Backend processes requests through route handlers and controllers, then interacts with a MySQL database using a connection pool. Authentication is enforced at the middleware layer using JWT verification before protected routes are accessed.

Architecture Diagram

FRONTEND [Browser]	BACKEND [Express.js]	DATABASE [MySQL]
HTML5 · CSS3 · JS Chart.js · Font Awesome Port: N/A	Node.js REST API Server JWT Auth Middleware Port: 5000	NexaBank DB MySQL Connection Pool 13 Tables

Data Flow Example:

1. User logs in via the Customer/Admin portal in the browser
2. app.js sends POST to http://localhost:5000/api/auth/login with credentials
3. Express verifies credentials via bcryptjs and returns a signed JWT
4. Subsequent requests include the JWT; middleware validates it before each route
5. Controllers execute parameterized SQL queries against MySQL
6. MySQL returns data rows; Express responds with JSON rendered dynamically in browser

5. Database Design

The database (NexaBank) contains thirteen tables linked through foreign keys, supporting a comprehensive banking domain model. The schema enforces referential integrity through cascading deletes and null-on-delete policies appropriate to each relationship.

Entity-Relationship (ER) Diagram

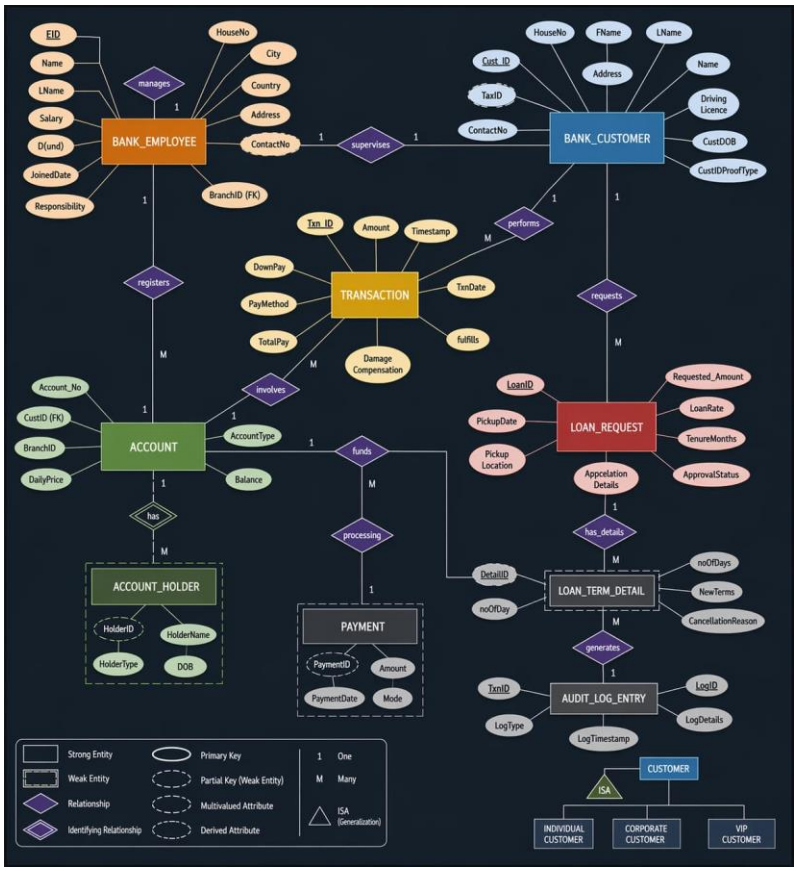

Figure: ER Diagram of the Banking Management System

Table	Primary Key / Key Columns	Foreign Key
BANK_EMPLOYEE	EID (PK), Name, LName, Email, Salary, Responsibility, BranchID	—
BANK_CUSTOMER	Cust_ID (PK), FName, LName, TaxID, KYCStatus, CustomerType, Email	—
ACCOUNT	Account_No (PK), AccountType, Balance, CreatedAt	CustID → BANK_CUSTOMER

Table	Primary Key / Key Columns	Foreign Key
ACCOUNT_HOLDER	HolderID (PK), HolderName, HolderType, Relationship	Account_No → ACCOUNT, Cust_ID → BANK_CUSTOMER
TRANSACTION	Txn_ID (PK), Amount, Transaction_Type, Description, PayMethod	CustID → BANK_CUSTOMER, Account_No → ACCOUNT
PAYMENT	PaymentID (PK), Amount, PaymentDate, Mode	Txn_ID → TRANSACTION
LOAN_REQUEST	LoanID (PK), Requested_Amount, LoanRate, TenureMonths, ApprovalStatus	Cust_ID → BANK_CUSTOMER
LOAN_TERM_DETAIL	DetailID (PK), noOfDays, NewTerms, CancellationReason	LoanID → LOAN_REQUEST
LOAN_EMI_SCHEDULE	EMI_ID (PK), DueDate, EMIAmount, PrincipalComponent, Status	LoanID → LOAN_REQUEST
FIXED_DEPOSIT	FD_ID (PK), Principal, InterestRate, TenureMonths, MaturityAmount	Cust_ID → BANK_CUSTOMER
RECURRING_DEPOSIT	RD_ID (PK), MonthlyInstallment, InterestRate, TenureMonths, MaturityAmount	Cust_ID → BANK_CUSTOMER
BENEFICIARY	Ben_ID (PK), Ben_Name, Ben_Account_No	Cust_ID → BANK_CUSTOMER
AUDIT_LOG_ENTRY	LogID (PK), LogType, LogTimestamp, LogDetails	TxnID → TRANSACTION, DetailID → LOAN_TERM_DETAIL
EMPLOYEE_SUPERVISES_CUSTOMER	EID + Cust_ID (Composite PK)	EID → BANK_EMPLOYEE, Cust_ID → BANK_CUSTOMER

Key Relationships

- BANK_CUSTOMER (1) --- ACCOUNT (Many)
- ACCOUNT (1) --- TRANSACTION (Many)

- BANK_CUSTOMER (1) --- LOAN_REQUEST (Many)
- LOAN_REQUEST (1) --- LOAN_EMI_SCHEDULE (Many)
- BANK_CUSTOMER (1) --- FIXED_DEPOSIT (Many)
- BANK_CUSTOMER (1) --- RECURRING_DEPOSIT (Many)
- BANK_EMPLOYEE (Many) --- BANK_CUSTOMER (Many) via EMPLOYEE_SUPERVISES_CUSTOMER

6. Modules and Features

6.1 Authentication Module

A login page with role-based access for Admin/Employee and Customer portals. Passwords are hashed using bcryptjs. Authentication uses JWT tokens issued on successful login and validated by middleware on every protected API route. Session tokens are stored in the browser's localStorage.

6.2 Dashboard Module

Displays summary statistics including total customers, accounts, transactions, active loans, and total balance. Uses Chart.js to render bar charts and line graphs for transaction trends and deposit summaries. All summary data is fetched in parallel using Promise.all for optimal performance.

6.3 Customer Management Module

Allows Admin to view, add, and manage bank customers. Includes KYC status tracking (PENDING / VERIFIED / REJECTED) and customer type classification (INDIVIDUAL, CORPORATE, VIP). Each customer can have multiple accounts, loans, FDs, and RDs.

6.4 Account & Transaction Module

Manages bank accounts with support for SAVINGS, CURRENT, and other account types. Tracks real-time balance. Transactions are logged as CREDIT or DEBIT with payment method details. Supports beneficiary-based fund transfers.

6.5 Loan Management Module

Customers can apply for loans with specified amount, interest rate, and tenure. Admin can APPROVE or REJECT loan requests. On approval, an EMI schedule is auto-generated using standard reducing balance calculations. Loan term details and cancellations are tracked separately.

6.6 Fixed Deposit & Recurring Deposit Module

Customers can open Fixed Deposits (lump sum) and Recurring Deposits (monthly installments). The system calculates and stores maturity dates and maturity amounts at the time of creation. Status is tracked as ACTIVE, MATURED, or CLOSED.

6.7 Role-Based Access Control

Admin/Employee role can manage employees, approve loans, verify KYC, and monitor all accounts. Customer role can view their own accounts, transactions, apply for loans, add beneficiaries, and manage FD/RD. Access is enforced server-side via JWT claims and middleware.

6.8 Audit Log Module

All significant system events (transactions, loan changes) are recorded in the AUDIT_LOG_ENTRY table with timestamps, log type, and details. This provides a traceable history of operations for compliance and debugging.

7. API Endpoints

Method	Endpoint	Description
POST	/api/auth/login	User login (Employee or Customer)
GET	/api/employees	List all bank employees
POST	/api/employees	Add a new bank employee
GET	/api/customers	List all customers (with KYC status)
POST	/api/customers	Register a new customer
GET	/api/accounts	List all accounts (with balance)
POST	/api/accounts	Open a new bank account
GET	/api/transactions	List all transactions
POST	/api/transactions	Post a new transaction (credit/debit)
GET	/api/loans	List all loan requests
POST	/api/loans	Submit a new loan application
PUT	/api/loans/:id/approve	Approve a loan request
PUT	/api/loans/:id/reject	Reject a loan request
GET	/api/fixed-deposits	List all fixed deposits
POST	/api/fixed-deposits	Open a new fixed deposit
GET	/api/recurring-deposits	List all recurring deposits
POST	/api/recurring-deposits	Open a new recurring deposit
GET	/api/beneficiaries	List beneficiaries for a customer
POST	/api/beneficiaries	Add a new beneficiary
GET	/api/payments	List all payments
GET	/api/audit	Fetch audit log entries
GET	/api/credit-cards	Credit card management
GET	/api/investments	Investment management

8. Implementation Highlights

- Server (Backend/server.js): Express app with 13 API route modules, JWT auth middleware, MySQL connection pool with auto-init, CORS, and environment-based configuration via dotenv.
- Database Init (Backend/config/db.js): Automatically creates the database and all tables on first run using a comprehensive schema (10,134-line DDL), ensuring zero-configuration setup for new developers.
- Frontend Logic (Frontend/app.js): 186,692-byte single-file frontend handling page routing, fetch-based API calls, dynamic table rendering, modal forms, Chart.js dashboards, toast notifications, and role-based UI injection.
- Styling (Frontend/styles.css): 63,087-byte stylesheet implementing a modern fintech-grade design with responsive layout, glassmorphism effects, gradient backgrounds, and custom scrollbars. Supplemented by mobile.css for responsive breakpoints.
- Security: All database queries use parameterized placeholders to prevent SQL injection. Passwords are hashed with bcryptjs (salt rounds = 10). JWT tokens are verified on every protected route via auth middleware.
- Loan EMI Generator (Backend/controllers/loanCalculator.js): Implements reducing-balance EMI computation and generates a full month-by-month repayment schedule stored in LOAN_EMI_SCHEDULE on loan approval.

9. Outputs

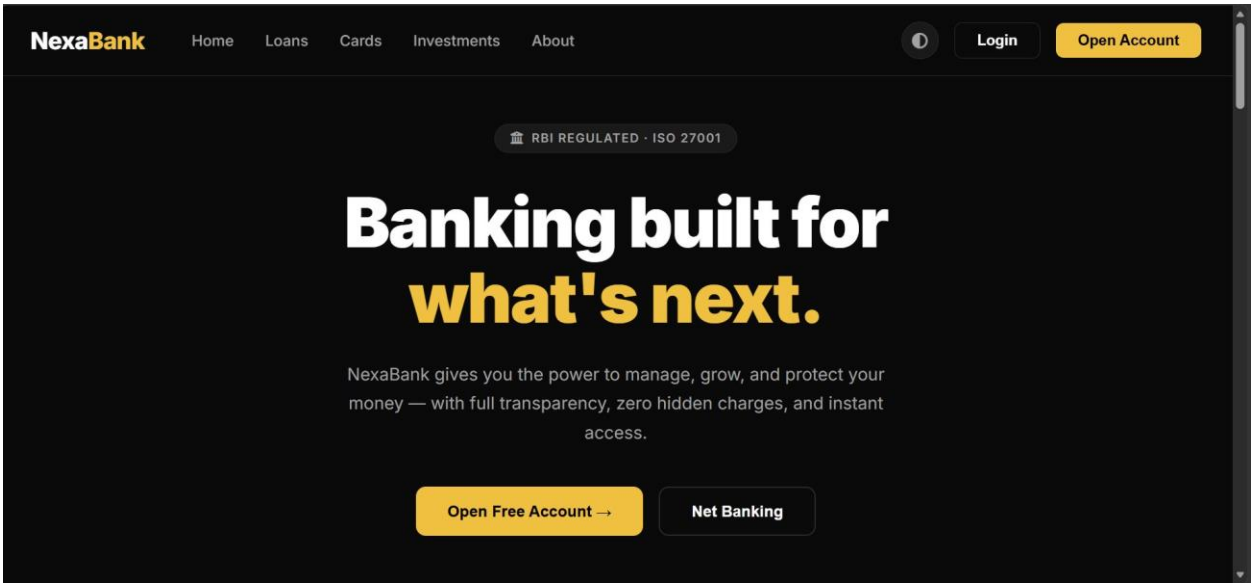

Figure 9.1: NexaBank Home Page — Banking built for what's next

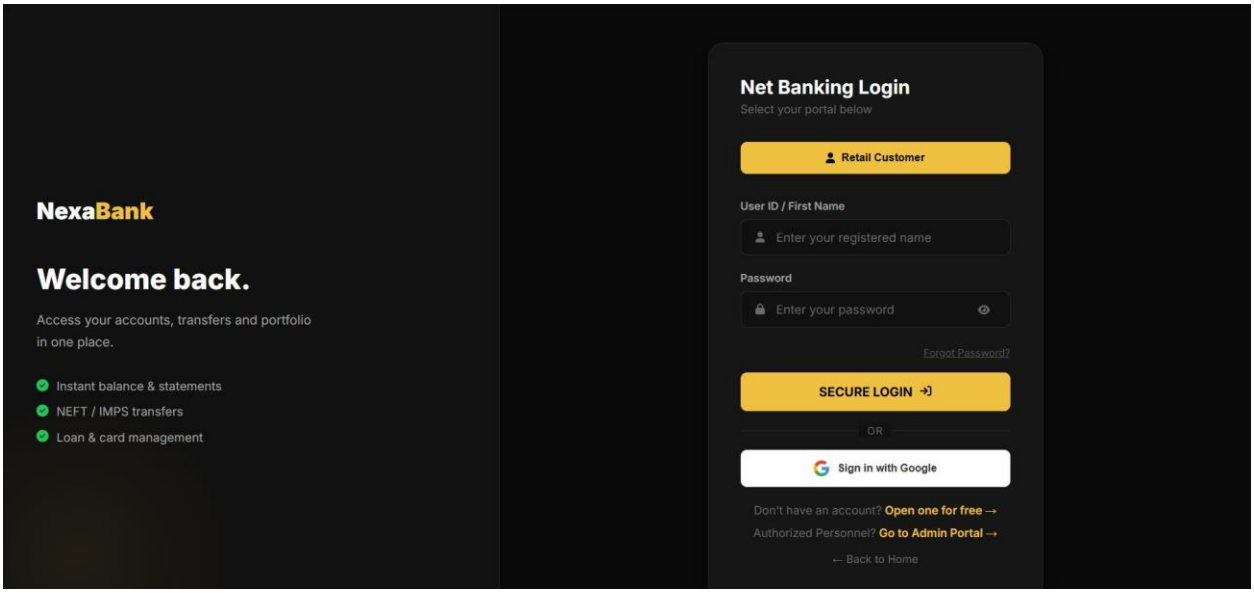

Figure 9.2: Net Banking Login — Customer Portal

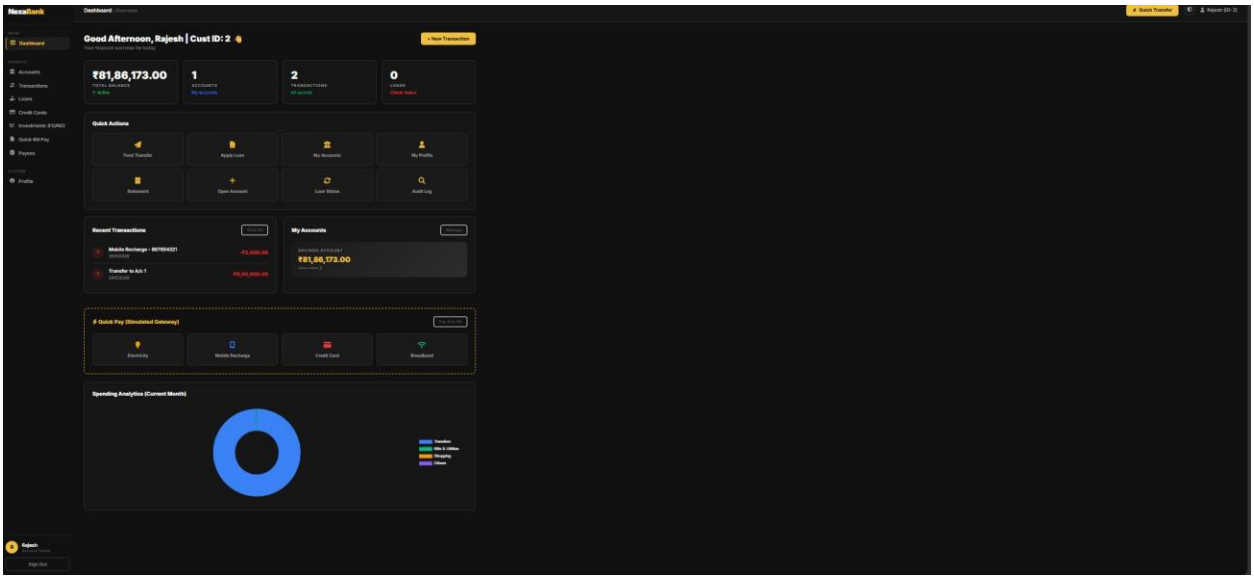

Figure 9.3: Customer Dashboard — Account Overview and Quick Actions

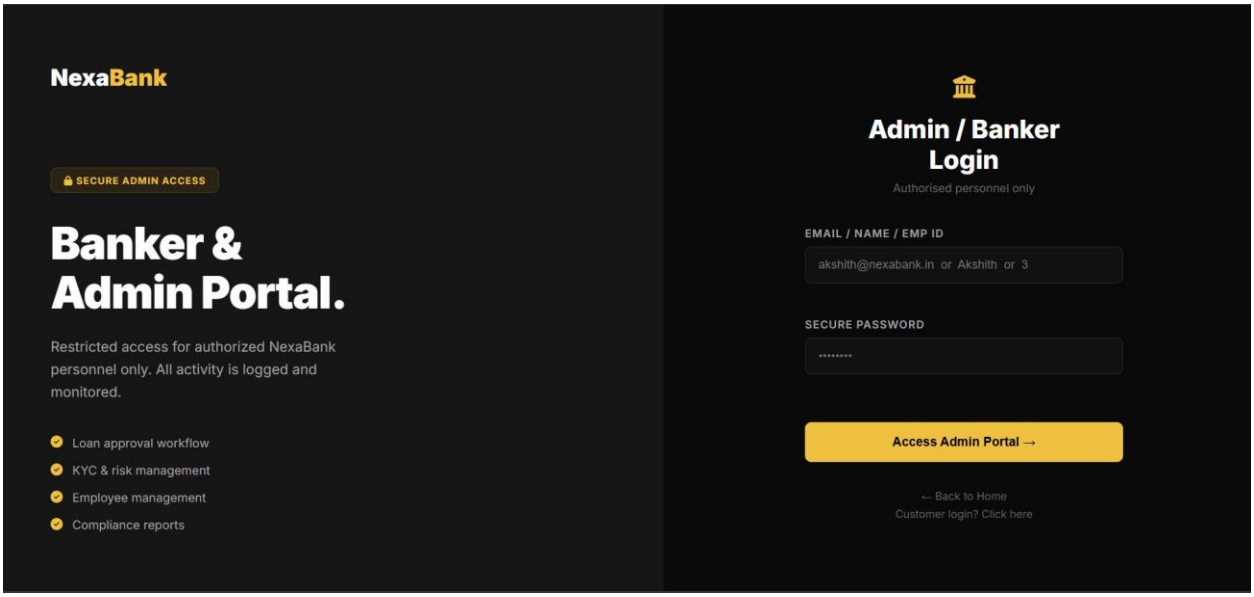

Figure 9.4: Admin / Banker Login Portal

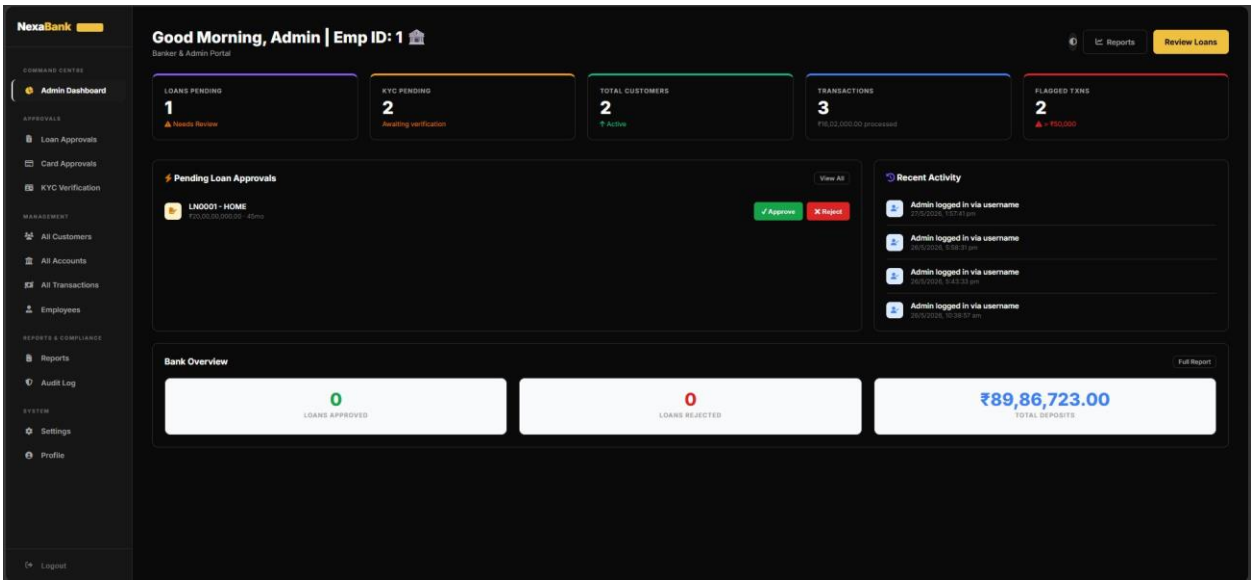

Figure 9.5: Admin Dashboard — Loan Approvals, KYC, and Bank Overview

10. Advantages

- Comprehensive banking domain coverage: accounts, loans, FD/RD, payments, audit, KYC
- Secure JWT-based authentication with bcrypt password hashing
- Automated EMI schedule generation on loan approval
- Real-time balance tracking and transaction history
- Role-based access control enforced both client-side and server-side
- SQL injection protection via fully parameterized queries
- Auto-initializing database schema requiring no manual setup

- Responsive, modern fintech-grade UI with Chart.js data visualizations

11. Future Enhancements

- Implement OTP-based two-factor authentication for customer login
- Add full CRUD support (PUT/DELETE) for all entity management pages
- Implement pagination and server-side search for large datasets
- Integrate a real payment gateway (Razorpay or Stripe) for live transactions
- Add appointment scheduling for branch visits and loan counseling
- Containerize the application with Docker for portable deployment
- Add unit and integration tests using Jest and Supertest
- Deploy on cloud infrastructure (AWS RDS + EC2 or Railway) with HTTPS

12. Conclusion

The Banking Management System successfully demonstrates a complete three-tier web application for managing financial institution data. The project applies core software engineering concepts including RESTful API design, relational database modeling, JWT-based authentication, connection pooling, role-based access control, and dynamic frontend rendering. With thirteen interrelated database tables and thirteen API route modules, the system covers a realistic banking domain end-to-end. The system is functional, secure, and serves as a solid foundation for a production-grade banking platform.

13. References

- Express.js Documentation — https://expressjs.com/
- MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/refman/8.0/en/
- mysql2 npm Package — https://www.npmjs.com/package/mysql2
- jsonwebtoken npm Package — https://www.npmjs.com/package/jsonwebtoken
- bcryptjs npm Package — https://www.npmjs.com/package/bcryptjs
- Chart.js Documentation — https://www.chartjs.org/docs/
- MDN Web Docs: Fetch API — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs: localStorage — https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
- Font Awesome Icons — https://fontawesome.com/
- Google Fonts — https://fonts.google.com/